70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · MT3

Sun'iy intellektga asoslangan zamonaviy kompyuter tizimlari arxitekturalarini (CPU, GPU, TPU, NPU va FPGA) qiyosiy tahlil qilish va ularning AI ilovalaridagi samaradorligini baholash

Bajarish bo‘yicha qo‘llanma: 7 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/MT3
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/MT3
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

Laboratoriya haqida

Bu mustaqil ishda siz besh xil hisoblash arxitekturasini analitik modellar va simulyatsiyalar orqali solishtirasiz: rasmiy cho'qqi qiymatlari bilan Roofline modeli, TPU sistolik massivining takt darajasidagi simulyatori, FPGA quvuri va fixed-point arifmetikasi, NPU uchun INT8 kvantlash, energiya va TCO kalkulyatori hamda yakuniy ko'p mezonli tanlov matritsasi. Real tezlatkichlar (GPU, TPU, NPU, FPGA) sandbox'da yo'q, shuning uchun ular modellar bilan baholanadi; yagona real o'lchov qurilmasi — 2 yadroli CPU, uning natijalari boshqa arxitekturalarga ekstrapolyatsiya qilinadi. Jadvaldagi barcha narx, quvvat va unumdorlik raqamlari taxminiy.

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, torchvision, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, shap, numba, psutil, pyyaml, fastapi, httpx.

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/MT3`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichda karkas (`s1_*.py`) berilgan: avval yo‘naltiruvchi savollarga o‘zingiz javob bering, so‘ng **TODO** joylarini to‘ldirib, `python3 s1_*.py` bilan ishga tushiring. Qiynalsangiz — qo‘llanma oxiridagi «Ishoralar» ilovasiga qarang. Natijani «Mening natijam» joyiga yozing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → MT3 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. Arxitekturalar pasporti va Roofline modeli
2. 2-qadam. TPU sistolik massivi: takt darajasidagi simulyator va MXU foydalanish modeli
3. 3-qadam. FPGA: quvur (pipeline), II va fixed-point aniqlik simulyatsiyasi
4. 4-qadam. NPU uchun INT8 kvantlash: aniqlik, hajm, kechikish va grafik bo'linishi
5. 5-qadam. Real CPU'da GEMM va konvolyutsiya o'lchovlari, boshqa arxitekturalarga ekstrapolyatsiya
6. 6-qadam. Energiya samaradorligi (TOP/s/Vt, mJ/tasvir) va 3 yillik TCO
7. 7-qadam. Yakuniy tanlov matritsasi: WSM, TOPSIS va vaznlar sezgirligi
8. Qo‘shimcha mashqlar (10)
9. Mustaqil topshiriqlar — 20 variant

1 1-qadam. Arxitekturalar pasporti va Roofline modeli

MAQSAD

CPU (AMD EPYC 9654), GPU (NVIDIA H100 SXM5), TPU (Google TPU v4), NPU (Hailo-8), chekka SoC (Jetson AGX Orin) va FPGA (AMD Alveo U250) uchun ochiq spetsifikatsiyalardagi cho'qqi qiymatlar (INT8/FP16 TOP/s, xotira o'tkazuvchanligi, quvvat) jadvali tuziladi. Har biri uchun ridge nuqtasi (cho'qqi / o'tkazuvchanlik) va TOP/s/Vt hisoblanadi. Keyin 6 ta tipik AI yadrosi (katta GEMM, ResNet conv3x3, BERT FFN, LLM dekod GEMV, depthwise conv, ReLU) uchun arifmetik intensivlik formuladan topiladi va Roofline bo'yicha erishiladigan unumdorlik hamda chegara turi (hisoblash yoki xotira) ko'rsatiladi. Log-log Roofline grafigi chiziladi. Barcha cho'qqi qiymatlari taxminiy — ular avlod, chastota va rejimga qarab o'zgaradi.

> **Kutilgan natija:** Pasport jadvali, AI jadvali (GEMM ~2700, GEMV ~2 op/bayt), Roofline jadvali: katta GEMM hamma joyda H, GEMV/ReLU hamma joyda X; LLM dekod token/s ~ BW/7 GB; grafik.

YO‘NALTIRUVCHI SAVOLLAR (SUQROT USULI)

Kod yozishdan oldin har bir savolga o‘zingiz qisqa javob yozing — javoblar sizni yechimga olib boradi. Ishoralar — qo‘llanma oxirida.

1. Roofline modelida qurilmaning qanday ikki «shifti» bor va ular qaysi ustunlardan (int8_tops, bw_gbs) olinadi?

2. Ridge nuqtasi nimani bildiradi: qaysi arifmetik intensivlikdan boshlab qurilma hisoblash chegarasiga o'tadi? H100 va Hailo-8 da u nega keskin farq qiladi?

3. M×K va K×N matritsalarni ko'paytirishda nechta amal (ko'paytirish + qo'shish) bajariladi va ideal holda nechta element xotiradan o'tadi?

4. Konvolyutsiyani GEMM ga o'xshatib qanday hisoblaysiz: amallarga k×k yadro qanday kiradi, baytlarga esa kirish, og'irlik va chiqish tenzorlari qanday? Depthwise da Co qayerga ketadi?

5. Erishiladigan unumdorlikni qanday topasiz va chegara turi (H yoki X) qaysi shartga qarab aniqlanadi?

6. Batch=1 da LLM dekodida har token uchun barcha og'irliklarni bir marta o'qish kerak. Unda token/s ni qaysi ikki kattalik belgilaydi?

7. Natijangizni qanday tekshirasiz: katta GEMM uchun AI ~2700, GEMV uchun ~2 op/bayt chiqdimi? Katta GEMM hamma qurilmada H, GEMV va ReLU esa X bo'ldimi?

KARKAS — TODO JOYLARINI TO‘LDIRING (FAYL: S1_*.PY)

```
# 1-qadam. Arxitekturalar pasporti va Roofline modeli: CPU, GPU, TPU, NPU, FPGA
# Cho'qqi qiymatlar ishlab chiqaruvchilarning ochiq spetsifikatsiyalaridan olingan, TAXMINIY
(zich, sparsity'siz).
import math
import pandas as pd
import matplotlib.pyplot as plt

pd.set_option("display.width", 160)

# --- Berilgan ma'lumotlar (o'zgartirmang) ---
NAN = float("nan")
QURILMA = [
    # nom,                        tur,    FP16/BF16 TFLOP/s, INT8 TOP/s, BW GB/s, quvvat Vt,
izoh
    ("AMD EPYC 9654 (96 yadro)",   "CPU",    7.4,    29.5,   460.8, 360, "AVX-512, 2,4 GHz"),
    ("NVIDIA H100 SXM5",           "GPU",  989.0,  1979.0,  3350.0, 700, "Tensor Core, zich;
HBM3"),
    ("Google TPU v4 (1 chip)",     "TPU",  275.0,   275.0,  1200.0, 200, "2 yadro x 4 MXU
128x128"),
    ("Hailo-8 (M.2)",              "NPU",    NAN,    26.0,     4.0, 2.5, "xost bilan PCIe ~4
GB/s"),
    ("Jetson AGX Orin 64GB",       "SoC",    NAN,   138.0,   204.8,  60, "GPU+DLA; zich ~138"),
    ("AMD Alveo U250",             "FPGA",   NAN,    33.3,    77.0, 225, "DDR4 4x"),
]
df = pd.DataFrame(QURILMA, columns=["nom", "tur", "fp16_tflops", "int8_tops", "bw_gbs", "vt",
"izoh"])

# TODO 1: ridge nuqtasi (op/bayt) va energiya samaradorligi (TOP/s/Vt) ustunlarini qo'shing
df["ridge_int8"] = None
df["tops_per_vt"] = None
print("=== Arxitekturalar pasporti (taxminiy, ochiq spetsifikatsiyalar) ===")
print(df[["nom", "tur", "int8_tops", "bw_gbs", "vt", "ridge_int8",
"tops_per_vt"]].to_string(index=False))

# TODO 2: amallar soni va baytlar (INT8, b=1 bayt/element) formulalarini yozing
def gemm(M, K, N, b=1):
    raise NotImplementedError("TODO 2: GEMM amallari va baytlari")

def conv(H, W, Ci, Co, k, b=1):
    raise NotImplementedError("TODO 2: conv amallari va baytlari")

def dwconv(H, W, C, k, b=1):
    raise NotImplementedError("TODO 2: depthwise conv amallari va baytlari")

YUKLAMA = {
    "GEMM":    ("GEMM 4096^3 (o'qitish bloki)",     gemm(4096, 4096, 4096)),
    "Conv3x3": ("ResNet conv3x3 56x56x64->64",      conv(56, 56, 64, 64, 3)),
    "BERT":    ("BERT FFN (128 token, 768->3072)",  gemm(128, 768, 3072)),
    "GEMV":    ("LLM dekod GEMV (1x4096x4096)",     gemm(1, 4096, 4096)),
    "DWconv":  ("Depthwise 3x3 112x112x32",         dwconv(112, 112, 32, 3)),
    "ReLU":    ("ReLU/qo'shish (element bo'yicha)", (112 * 112 * 64, 2 * 112 * 112 * 64)),
}
print("\n=== Yuklamalar: arifmetik intensivlik ===")
# TODO 3: har yuklama uchun AI = amal / bayt ni chiqaring

print("\n=== Roofline: erishiladigan INT8 unumdorlik, TOP/s (chegara turi) ===")
# TODO 4: har qurilma x yuklama uchun P = min(cho'qqi, AI*BW) va chegara turini (H/X) jadvalga
yig'ing

print("\n=== LLM dekod (7B, INT8, batch=1): token/s ~ BW / 7 GB ===")
# TODO 5: har qurilma uchun token/s ni baholang
```

```
# TODO 6: log-log Roofline grafigini chizing (ax.loglog)
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Arxitekturalar pasporti`, `=== Roofline: erishiladigan INT8`, `=== LLM dekod`

TAJRIBA QILIB KO‘RING

- Jadvalga NVIDIA A100 yoki L4 ni qo'shing (ma'lumot varag'idan qiymatlarni o'zingiz toping) va ridge nuqtasini solishtiring.
- BERT FFN da token sonini 128 dan 2048 ga oshiring — H100 qachon hisoblash chegarasiga o'tadi?
- Yuklamalarni FP16 (2 bayt/element) bilan qayta hisoblang: AI va chegara turi qanday o'zgaradi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. TPU sistolik massivi: takt darajasidagi simulyator va MXU foydalanish modeli

MAQSAD

NumPy'da output-stationary sistolik massiv takt-ma-takt simulyatsiya qilinadi: A satrlari chapdan kechikish (skew) bilan, B ustunlari yuqoridan oqib kiradi, registrlar har taktda siljiydi. Natija A@B bilan aynan mos kelishi, taktlar soni K+R+C-2 formulasiga tengligi va PE foydalanish ulushi tekshiriladi; t=6 dagi diagonal to'lqin fronti ko'rsatiladi. So'ngra TPU v4 ning 128x128 MXU (8 dona, 1,05 GHz = 275 TFLOP/s) uchun weight-stationary analitik model bilan real qatlamlarda (BERT, ResNet, LLM dekod, toq o'lcham) foydalanish va samarali TFLOP/s hisoblanadi.

> **Kutilgan natija:** Simulyatorda xato 0; K=256 da foydalanish ~90 %; MXU: BERT 4096 token ~94 %, GEMV M=1 ~0,4 %, toq N=130 ~45 %; grafik.

YO‘NALTIRUVCHI SAVOLLAR (SUQROT USULI)

Kod yozishdan oldin har bir savolga o‘zingiz qisqa javob yozing — javoblar sizni yechimga olib boradi. Ishoralar — qo‘llanma oxirida.

1. Output-stationary massivda nima «turib qoladi» va nima oqadi? A va B qaysi tomonlardan, qaysi yo'nalishda harakatlanadi?

2. A[i, k] va B[k, j] PE(i, j) ga bir vaqtda yetib kelishi uchun i-satr va j-ustun necha takt kechikib kirishi kerak? Bundan oxirgi PE(R-1, C-1) uchun oxirgi taktni qanday chiqarasiz?

3. Registrlarni bir taktga siljitishni NumPy kesmalari bilan qanday yozasiz va nega .copy() kerak bo'lishi mumkin?

4. Kirishdagi indeks k = t − i chegaradan tashqarida bo'lsa, PE ga nima berilishi kerak? Faol PE ni qanday aniqlaysiz?

5. Weight-stationary MXU da bitta 128×128 og'irlik plitkasi uchun necha takt ketadi va K×N matritsa nechta plitkaga bo'linadi?

6. Natijani qanday tekshirasiz: simulyator xatosi 0 mi, taktlar K+R+C−2 ga tengmi, 16×16 K=256 da foydalanish ~90 % chiqdimi?

KARKAS — TODO JOYLARINI TO‘LDIRING (FAYL: S2_*.PY)

```
# 2-qadam. TPU yuragi — sistolik massiv: takt darajasidagi simulyator va MXU 128x128
foydalanish modeli
# Chap tomondan A satrlari (kechikish bilan, "skew"), yuqoridan B ustunlari oqib kiradi; har PE
har taktda
```

```
# bitta MAC bajaradi va ma'lumotni qo'shnisiga uzatadi. Natija PE ichida to'planadi (output-
stationary).
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)


def sistolik(A, B, R, C, kadr_t=None):
    """R x C sistolik massiv, output-stationary. A: (R, K), B: (K, C). Qaytaradi: natija,
taktlar, foydali MAC, kadr."""
    K = A.shape[1]
    a_reg = np.zeros((R, C)); b_reg = np.zeros((R, C)); acc = np.zeros((R, C))
    faol_mac = 0; kadr = None
    # TODO 1: umumiy taktlar soni T ni toping (oxirgi ko'paytma qaysi PE ga qachon keladi?)
    T = None
    for t in range(T):
        # TODO 2: registrlarni siljiting (a — o'ngga, b — pastga)
        # TODO 3: chap ustunga A satrlarini, yuqori satrga B ustunlarini kechikish (skew) bilan
kiriting
        # TODO 4: faol PE'larni sanang va acc ga a_reg*b_reg ni qo'shing; kadr_t da faol
niqobni saqlang
        raise NotImplementedError("TODO 2-4: bir taktlik sistolik qadam")
    return acc, T, faol_mac, kadr


print("=== Sistolik massiv simulyatori: to'g'rilik tekshiruvi ===")
for R, C, K in [(4, 4, 4), (8, 8, 8), (8, 8, 64), (16, 16, 16), (16, 16, 256)]:
    A = rng.integers(1, 9, (R, K)).astype(float); B = rng.integers(1, 9, (K, C)).astype(float)
    D, T, mac, _ = sistolik(A, B, R, C)
    # TODO 5: xato = |D - A@B| maks., PE-foydalanish = mac / (R*C*T) ni chiqaring

# TODO 6: 8x8 massivda t=6 kadrini '#'/'.' ko'rinishida chiqaring

# --- TPU v4 MXU modeli: 128x128, weight-stationary; 4 MXU x 2 yadro, 1,05 GHz ---
RM = CM = 128
F = 1.05e9; MXU = 8


def mxu_foydalanish(M, K, N, ikki_bufer=True):
    """Og'irlik plitkasi (128x128) yuklanadi, M ta faollashtirish satri oqib o'tadi.
    Har plitka: yuklash 128 takt (ikki buferda yashiriladi) + M + 2*128 - 2 takt. Bo'sh
kataklar (padding) — isrof."""
    # TODO 7: plitkalar soni, jami taktlar va foydali MAC / (RM*CM*jami_takt) ni qaytaring
    raise NotImplementedError("TODO 7: MXU foydalanish modeli")


QATLAM = [  # nom, M (token/piksel x batch), K, N
    ("BERT FFN, 512 token",            512, 768, 3072),
    ("BERT FFN, 4096 token (batch 8)", 4096, 768, 3072),
    ("BERT FFN, 32 token",             32, 768, 3072),
    ("ResNet conv1 (K=7*7*3=147)",     112 * 112, 147, 64),
    ("ResNet conv3x3 (K=576, N=64)",   56 * 56, 576, 64),
    ("ResNet conv3x3 (K=4608, N=512)", 7 * 7 * 32, 4608, 512),
    ("LLM dekod GEMV (M=1)",           1, 4096, 4096),
    ("LLM dekod, batch 64",            64, 4096, 4096),
    ("Toq o'lcham (N=130)",            2048, 1024, 130),
]
print("\n=== TPU v4 MXU foydalanish modeli (analitik, taxminiy) ===")
# TODO 8: har qatlam uchun foydalanish_% va samarali_TFLOP/s (275 * u) jadvalini chiqaring,
grafik chizing
```

```
print("\n=== Simulyatsiya tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Sistolik massiv simulyatori`, `=== TPU v4 MXU foydalanish modeli`, `=== Simulyatsiya tugadi ===`

TAJRIBA QILIB KO‘RING

- Massivni weight-stationary rejimga o'zgartiring (B oldindan PE'larga yuklanadi) va taktlar formulasini chiqaring.
- MXU o'lchamini 256x256 ga oshiring (TPU v6e kabi) — kichik qatlamlarda foydalanish qanday o'zgaradi?
- Toq o'lcham N=130 uchun 128 ga moslash (padding) va 2 ta plitka o'rniga N ni 128 ga qisqartirish samarasini solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. FPGA: quvur (pipeline), II va fixed-point aniqlik simulyatsiyasi

MAQSAD

Birinchi qismda 4 bosqichli MAC quvuri vaqt diagrammasi bilan simulyatsiya qilinadi: initiation interval (II) 1, 2, 4 bo'lganda o'tkazuvchanlik qanday tushishi ko'rsatiladi, Alveo U250 (~12288 DSP, 300 MHz) uchun INT16/INT8/FP32 rejimlarda TOP/s bahosi chiqariladi. Ikkinchi qismda digits ustida o'qitilgan MLP 64-32-10 butunlay butun sonli (fixed-point) arifmetikada bajariladi: Qm.n formatlari, to'yinish (saturation), siljitish orqali masshtab va akkumulyator kengligi. Har format uchun aniqlik, SQNR va to'yingan og'irliklar soni jadvalda.

> **Kutilgan natija:** II=1 da 8 kirish 11 taktda; INT8 packing ~12 TOP/s (taxminiy); 16 bitda aniqlik suzuvchi nuqta bilan bir xil (~0,97), Q0.7 da to'yinish va keskin tushish, 12 bitli akkumulyatorda ~0,6.

YO‘NALTIRUVCHI SAVOLLAR (SUQROT USULI)

Kod yozishdan oldin har bir savolga o‘zingiz qisqa javob yozing — javoblar sizni yechimga olib boradi. Ishoralar — qo‘llanma oxirida.

1. 4 bosqichli quvurda har bosqich 1 takt oladi. Agar yangi kirish har II taktda qabul qilinsa, i-kirish qachon boshlanadi va qachon tugaydi?

2. II=1 da 8 ta kirish necha taktda tugashi kerak va bu quvursiz (har kirish 4 takt, ketma-ket) holatdan necha marta tez?

3. FPGA ning TOP/s bahosi qaysi omillarga bog'liq: DSP soni, band ulushi, bitta DSP dagi MAC soni, chastota va II qanday birlashadi?

4. Qm.n formatda haqiqiy sonni butun songa qanday o'tkazasiz va diapazondan chiqqan qiymat bilan nima qilinadi?

5. x (Q kasr_a) va w (Q kasr_w) ko'paytmasining kasr qismi necha bit bo'ladi? Bias'ni akkumulyatorga qo'shishdan oldin qanday masshtablash kerak, keyingi qatlamga o'tishda esa nima uchun o'ngga siljitish (>>) ishlatiladi?

6. Akkumulyator kengligi (akk_bit) tor bo'lsa nima yuz beradi? SQNR ni qaysi ikki signaldan hisoblaysiz?

7. Natijani tekshiring: 16 bitda aniqlik suzuvchi nuqta bilan bir xil (~0,97) chiqdimi? Q0.7 da to'yingan og'irliklar paydo bo'ldimi, 12 bitli akkumulyatorda aniqlik ~0,6 gacha tushdimi?

KARKAS — TODO JOYLARINI TO‘LDIRING (FAYL: S3_*.PY)

```
# 3-qadam. FPGA: quvur (pipeline) va fixed-point aniqlik simulyatsiyasi
# (a) MAC yo'li 4 bosqichga bo'lingan (o'qish -> ko'paytirish -> qo'shish -> yozish). II —
yangi kirish oralig'i.
# (b) digits MLP og'irlik va faollashtirishlarini Qm.n formatga keltirib, to'yinishli butun
sonli arifmetikada bajaramiz.
import warnings; warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

# ---------------- (a) Quvur simulyatori ----------------
def quvur(n_kirish, bosqich=4, II=1):
    """Qaytaradi: jami taktlar, har kirishning boshlanish takti."""
    # TODO 1: har kirish qachon boshlanadi va oxirgisi qachon tugaydi?
    raise NotImplementedError("TODO 1: quvur vaqt modeli")


print("=== (a) Quvur: 4 bosqichli MAC, 8 ta kirish ===")
# TODO 2: II = 1, 2, 4 uchun jami taktlar va o'tkazuvchanlikni (natija/takt) chiqaring; II=1
vaqt diagrammasini chizing

print("\n=== FPGA o'tkazuvchanlik modeli (taxminiy) ===")
DSP, F = 12288, 300e6
REJIM = [("INT16, 1 MAC/DSP", 1, 0.8, 1), ("INT8, 2 MAC/DSP (packing)", 2, 0.8, 1),
         ("INT8, II=2 (xotira cheklovi)", 2, 0.8, 2), ("FP32 (~3 DSP/MAC)", 1 / 3, 0.8, 1)]
# TODO 3: har rejim uchun TOP/s = DSP * ulush * MAC/DSP * 2 op * F / II ni hisoblang

# ---------------- (b) Fixed-point aniqlik ----------------
X, y = load_digits(return_X_y=True); X = (X / 16.0)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
mlp = MLPClassifier(hidden_layer_sizes=(32,), max_iter=300, random_state=0).fit(Xtr, ytr)
W1, W2 = mlp.coefs_; b1, b2 = mlp.intercepts_
acc_fp = (mlp.predict(Xte) == yte).mean()


def qkvant(x, butun, kasr, jami):
    """Ishorali fixed-point: jami bit, 'kasr' bit kasr qism. Qaytaradi: butun sonlar,
to'yinganlar soni."""
    # TODO 4: 2^kasr ga ko'paytirib yaxlitlang va [-2^(jami-1), 2^(jami-1)-1] ga to'yintiring
    raise NotImplementedError("TODO 4: Qm.n kvantlash")


def fixed_mlp(X, jami, kasr_w, kasr_a, akk_bit):
    """Butun sonli inferens. Qaytaradi: aniqlik, SQNR (dB), to'yingan og'irliklar soni."""
    # TODO 5: x, W1, W2 ni kvantlang; bias'ni Q(kasr_a+kasr_w) ga keltiring; akkumulyatorni
akk_bit da to'yintiring
    # TODO 6: ReLU dan keyin kasr_w bitga o'ngga siljitib masshtabni qaytaring, ikkinchi
qatlamni hisoblang
    # TODO 7: suzuvchi nuqta etalon bilan SQNR = 10*log10(signal / shovqin) ni toping
    raise NotImplementedError("TODO 5-7: fixed-point MLP")


print(f"\n=== (b) Fixed-point MLP 64-32-10 (digits), suzuvchi nuqta aniqligi: {acc_fp:.4f}
===")
for jami, kw, ka, akk in [(16, 12, 12, 48), (12, 8, 8, 32), (8, 6, 6, 24), (8, 4, 5, 24), (8,
7, 7, 24),
                          (6, 4, 4, 20), (4, 2, 2, 16), (8, 6, 6, 12)]:
    a, sq, sat = fixed_mlp(Xte, jami, kw, ka, akk)
    # TODO 8: natijalarni (bit, Q_w, Q_a, akk_bit, aniqlik, SQNR, to'yingan_w) jadvalga yig'ib
```

```
chiqaring

print("\n=== FPGA simulyatsiyasi tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== (a) Quvur`, `=== FPGA o'tkazuvchanlik modeli`, `=== (b) Fixed-point MLP`, `=== FPGA simulyatsiyasi tugadi ===`

TAJRIBA QILIB KO‘RING

- Quvurga 2 taktli xotira o'qish bosqichini qo'shing (5 bosqich) — kechikish va o'tkazuvchanlik qanday o'zgaradi?
- Yashirin qatlamni 128 neyronga oshiring: 8 bitli akkumulyator yetarli bo'lishi uchun nechta bit kerak? (log2(K) qoidasi)
- Har qatlamga alohida Q format tanlang (qatlam bo'yicha diapazon) va aniqlikni solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. NPU uchun INT8 kvantlash: aniqlik, hajm, kechikish va grafik bo'linishi

MAQSAD

Kichik CNN digits ustida o'qitilib, PyTorch eager statik kvantlash (qnnpack) bilan INT8 ga o'tkaziladi: conv+relu birlashtiriladi, per-tensor va per-channel og'irlik kvantlash, 8 va 256 namunali kalibrlash solishtiriladi. Aniqlik, model hajmi va batch 1/64 kechikishi o'lchanadi. So'ng har qatlam og'irliklarining simmetrik INT8 SQNR'i hisoblanadi. Oxirida NPU qo'llamaydigan operatorlar (SiLU, NMS) CPU'ga qaytganda PCIe orqali qatnov kechikishi modellashtiriladi va SiLU -> ReLU almashtirish samarasi ko'rsatiladi.

> **Kutilgan natija:** FP32 va INT8 aniqligi deyarli teng (~0,95), hajm ~3,5x kichik; CPU'da batch=1 da INT8 tez bo'lmasligi mumkin, batch 64 da tezroq; SQNR ~40-47 dB; bo'linish modelida o'tishlar 5 -> 3, kechikish ~3,0 -> 2,4 ms.

YO‘NALTIRUVCHI SAVOLLAR (SUQROT USULI)

Kod yozishdan oldin har bir savolga o‘zingiz qisqa javob yozing — javoblar sizni yechimga olib boradi. Ishoralar — qo‘llanma oxirida.

1. Statik kvantlashda og'irliklardan tashqari yana nima INT8 ga o'tadi va buning uchun nega kalibrlash ma'lumoti kerak?

2. Model hajmini qanday o'lchaysiz va FP32 → INT8 da u nazariy jihatdan necha marta kamayishi kerak? Nega amalda ~3,5x chiqadi?

3. Eager rejimida kvantlashning uch bosqichi qaysi tartibda keladi va conv+relu ni birlashtirish (fuse) nima uchun foydali?

4. Per-tensor va per-channel og'irlik kvantlash farqi nimada? Qaysi biri kanallar diapazoni har xil bo'lganda aniqroq?

5. Bitta qatlam uchun simmetrik INT8 SQNR ni qanday hisoblaysiz? Masshtab nima bo'ladi va qaysi diapazonga klip qilinadi?

6. Grafik bo'linishi modelida kechikish nimalardan yig'iladi? Qachon PCIe nusxalash vaqti qo'shiladi?

7. Natijalaringiz kutilganga mosmi: FP32 va INT8 aniqligi ~0,95, batch=64 da INT8 tezroq, o'tishlar 5 → 3 va kechikish ~3,0 → 2,4 ms?

KARKAS — TODO JOYLARINI TO‘LDIRING (FAYL: S4_*.PY)

```
# 4-qadam. NPU uchun INT8 kvantlash: aniqlik, hajm, kechikish va operatorlarni bo'lish
(partitioning)
# Kichik CNN'ni PyTorch statik kvantlash (qnnpack) bilan INT8 ga o'tkazamiz: per-tensor vs per-
channel,
# kalibrlash hajmi ta'siri va NPU qo'llamaydigan operatorlar CPU'ga qaytganda (fallback)
yo'qotilgan vaqt.
import warnings; warnings.filterwarnings("ignore")
import copy, io, time
import numpy as np
import torch, torch.nn as nn
import torch.ao.quantization as tq
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0); torch.set_num_threads(2)
torch.backends.quantized.engine = "qnnpack"      # AVX'siz CPU uchun
torch.backends.nnpack.set_flags(False)

X, y = load_digits(return_X_y=True); X = (X / 16.0).astype(np.float32).reshape(-1, 1, 8, 8)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
Ttr, Tytr, Tte = torch.tensor(Xtr), torch.tensor(ytr), torch.tensor(Xte)


class CNN(nn.Module):
    def __init__(s):
        super().__init__()
        s.quant = tq.QuantStub(); s.dequant = tq.DeQuantStub()
        s.c1 = nn.Conv2d(1, 16, 3, padding=1); s.r1 = nn.ReLU()
        s.c2 = nn.Conv2d(16, 32, 3, padding=1); s.r2 = nn.ReLU(); s.p = nn.MaxPool2d(2)
        s.fc1 = nn.Linear(32 * 4 * 4, 64); s.r3 = nn.ReLU(); s.fc2 = nn.Linear(64, 10)

    def forward(s, x):
        x = s.quant(x)
        x = s.p(s.r2(s.c2(s.r1(s.c1(x)))))
        x = s.r3(s.fc1(x.reshape(x.shape[0], -1)))
        return s.dequant(s.fc2(x))


m = CNN(); opt = torch.optim.Adam(m.parameters(), 2e-3); lf = nn.CrossEntropyLoss()
for ep in range(6):
    perm = torch.randperm(len(Ttr))
    for i in range(0, len(Ttr), 64):
        idx = perm[i:i + 64]; opt.zero_grad(); lf(m(Ttr[idx]), Tytr[idx]).backward();
opt.step()
m.eval()

# TODO 1: aniqlik(model), hajm_kb(model) (state_dict -> BytesIO) va kechikish_ms(model, batch)
(median, ms) yozing
def aniqlik(model): raise NotImplementedError("TODO 1")
def hajm_kb(model): raise NotImplementedError("TODO 1")
def kechikish_ms(model, batch=1, n=300): raise NotImplementedError("TODO 1")


def kvantla(model, qconfig, n_kalib):
    # TODO 2: nusxa oling, qconfig bering, conv+relu / linear+relu ni birlashtiring
(fuse_modules)
    # TODO 3: prepare -> n_kalib namuna bilan kalibrlash -> convert
    raise NotImplementedError("TODO 2-3: statik kvantlash")


# TODO 4: per-tensor (MinMaxObserver) va per-channel (PerChannelMinMaxObserver) QConfig'larini
tuzing
per_tensor = None
```

```
per_channel = None

print("=== INT8 statik kvantlash (qnnpack, CPU — NPU o'rniga) ===")
print(f"{'FP32':32s} {aniqlik(m):8.4f} {hajm_kb(m):8.1f} {kechikish_ms(m):8.3f}
{kechikish_ms(m, 64, 100):8.3f}")
# TODO 5: 2 xil qconfig x kalibrlash (8, 256) uchun aniqlik, hajm, b=1 va b=64 kechikishini
chiqaring

print("\n=== Qatlam og'irliklari: INT8 kvantlash xatosi (SQNR, dB) ===")
# TODO 6: har 2D+ parametr uchun simmetrik INT8 (masshtab = max|w|/127) va SQNR ni hisoblang

print("\n=== NPU grafik bo'linishi modeli (taxminiy, Hailo-8 kabi M.2 NPU) ===")
NPU_TOPS, PCIE_GBS, CPU_GOPS = 26 * 0.3, 3.0, 50   # NPU real foydalanish ~30 %, PCIe ~3 GB/s,
xost CPU ~50 GOP/s
OPLAR = [  # (nom, amallar GOP, chiqish tenzori MB, NPU qo'llaydimi)
    ("conv_stem", 0.9, 1.6, True), ("conv_blok1", 2.1, 0.8, True), ("SiLU (qo'llanmaydi)",
0.01, 0.8, False),
    ("conv_blok2", 2.4, 0.4, True), ("NMS/postprocess", 0.05, 0.05, False), ("conv_head", 0.6,
0.2, True)]
# TODO 7: grafikni ketma-ket yurib, har CPU<->NPU o'tishda mb/PCIE_GBS qo'shing; SiLU -> ReLU
variantini solishtiring

print("\n=== Kvantlash tahlili tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== INT8 statik kvantlash`, `=== Qatlam og'irliklari`, `=== NPU grafik bo'linishi modeli`, `=== Kvantlash tahlili tugadi ===`

TAJRIBA QILIB KO‘RING

- Kalibrlash uchun HistogramObserver ishlating va aniqlik farqini o'lchang.
- Modelga SiLU qo'shing va kvantlangan model qaysi qatlamda dequant/quant qilishini ko'ring (print(q)).
- PCIE_GBS ni 0,5 GB/s (USB) ga o'zgartiring — Edge TPU USB kabi qurilmada bo'linish narxi qancha?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. Real CPU'da GEMM va konvolyutsiya o'lchovlari, boshqa arxitekturalarga ekstrapolyatsiya

MAQSAD

Sandbox CPU (2 yadro) uchun haqiqiy Roofline quriladi: xotira o'tkazuvchanligi STREAM triad bilan, amaliy cho'qqi esa eng yaxshi FP32 GEMM bilan o'lchanadi. GEMM 64...1024, GEMV, conv3x3, conv1x1, depthwise va ReLU uchun GFLOP/s, arifmetik intensivlik va Roofline'ga nisbatan samaradorlik jadvali chiqadi. Keyin har yadro samaradorligi saqlanadi deb faraz qilinib, 1-qadam pasportidagi arxitekturalarda kutilgan vaqt (ishga tushirish xarajati bilan) ekstrapolyatsiya qilinadi. Bu — birinchi yaqinlashish, aniq bashorat emas.

> **Kutilgan natija:** Triad ~20-40 GB/s, GEMM cho'qqisi ~40-60 GFLOP/s (mashinaga bog'liq); depthwise samaradorligi < 10 %; ekstrapolyatsiyada kichik yadrolar tezlatkichlarda 5-10 us 'pol'ga uriladi.

YO‘NALTIRUVCHI SAVOLLAR (SUQROT USULI)

Kod yozishdan oldin har bir savolga o‘zingiz qisqa javob yozing — javoblar sizni yechimga olib boradi. Ishoralar — qo‘llanma oxirida.

1. Nega 1-qadamdagi ishlab chiqaruvchi raqamlari o'rniga sandbox CPU ning cho'qqisini va o'tkazuvchanligini o'zimiz o'lchaymiz? Bu ikki kattalik qaysi testlardan olinadi?

2. Vaqtni ishonchli o'lchash uchun nima qilish kerak: birinchi chaqiruvlarni hisobga olish kerakmi, o'rtacha yoki median yaxshiroqmi?

3. Triad a = b + s·c da xotiradan nechta massiv o'tadi? n = 4M float32 bo'lsa, bayt sonini qanday hisoblaysiz?

4. Conv2d uchun amallar va baytlar formulasini yozing: groups=g (depthwise) bo'lganda Ci qanday o'zgaradi?

5. Samaradorlik = o'lchangan / roofline. Nima uchun ba'zi yadrolarda u 100 % dan oshib ketishi mumkin?

6. Ekstrapolyatsiyada FP16/BF16 rejimdagi arxitekturalar uchun bayt soni qanday o'zgaradi va ishga tushirish xarajati (us) kichik yadrolarda qanday rol o'ynaydi?

7. Natijangizni tekshiring: triad ~20–40 GB/s, GEMM cho'qqisi ~40–60 GFLOP/s oralig'idami? Qiymatlar bundan keskin farq qilsa, qaysi birliklarni tekshirasiz?

KARKAS — TODO JOYLARINI TO‘LDIRING (FAYL: S5_*.PY)

```
# 5-qadam. Real CPU'da GEMM va konvolyutsiya o'lchovlari, o'lchangan Roofline va boshqa arxitekturalarga ekstrapolyatsiya
# Avval sandbox CPU ning haqiqiy cho'qqisi (eng yaxshi GEMM) va xotira o'tkazuvchanligi (STREAM triad) o'lchanadi,
# keyin har yadro samaradorligi topiladi va shu samaradorlik bilan GPU/TPU/NPU/FPGA'dagi vaqt TAXMINIY bashorat qilinadi.
import time
import numpy as np
import pandas as pd
import torch
import torch.nn.functional as Fn
import matplotlib.pyplot as plt

torch.manual_seed(0); torch.set_num_threads(2); torch.backends.nnpack.set_flags(False)


def olch(f, takror=7):
    # TODO 1: isitish (warm-up) chaqiruvlaridan keyin takror marta o'lchab, median vaqtni (s) qaytaring
    raise NotImplementedError("TODO 1: vaqt o'lchash")


print("=== Sandbox CPU o'lchovlari ===")
n = 4 * 1024 * 1024
b = torch.rand(n); c = torch.rand(n); a = torch.empty(n)
# TODO 2: STREAM triad a = b + 1.5*c ni o'lchab, BW (GB/s) ni hisoblang (nechta massiv x necha bayt?)
BW = None

natija = []   # (yadro nomi, amallar, baytlar, vaqt_s)
# TODO 3: GEMM N = 64..1024 va GEMV 4096x2048 uchun ops, bayt (FP32 = 4 bayt) va vaqtni natijaga qo'shing
# TODO 4: conv3x3 32->64 28x28, conv1x1 64->128 28x28, depthwise 3x3 64ch 56x56 (batch 8) va ReLU 8x64x56x56 ni o'lchang

df = pd.DataFrame(natija, columns=["yadro", "ops", "bayt", "t_s"])
# TODO 5: GFLOP/s, AI, PEAK (amaliy cho'qqi), roofline = min(PEAK, AI*BW) va samaradorlik_% ustunlarini qo'shing
PEAK = None
print(f"Amaliy cho'qqi (eng yaxshi GEMM): {PEAK:6.1f} GFLOP/s, ridge nuqtasi = {PEAK / BW:5.1f} FLOP/bayt\n")

ARX = {  # nom: (cho'qqi TOP/s, BW GB/s, ishga tushirish xarajati us — TAXMINIY faraz)  — 1-qadam pasportidan
    "EPYC 9654 (INT8)": (29.5, 460.8, 0), "H100 (FP16)": (989.0, 3350.0, 5), "TPU v4 (BF16)": (275.0, 1200.0, 5),
    "Jetson AGX Orin (INT8)": (138.0, 204.8, 8), "Alveo U250 (INT8)": (33.3, 77.0, 10), "Hailo-8 (INT8)": (26.0, 4.0, 50)}
print("\n=== Ekstrapolyatsiya: kutilgan vaqt, mikrosoniya (samaradorlik sandbox'dagidek + ishga tushirish xarajati) ===")
# TODO 6: har yadro x arxitektura uchun vaqt = ust + ops / (e * min(cho'qqi, AI_yangi * BW)); e 5..100 % ga cheklanadi

# TODO 7: o'lchangan Roofline grafigini chizing (chiziq + nuqtalar)
print("\n=== O'lchovlar tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Sandbox CPU o'lchovlari ===`, `Amaliy cho'qqi`, `=== Ekstrapolyatsiya`, `=== O'lchovlar tugadi ===`

TAJRIBA QILIB KO‘RING

- torch.set_num_threads(1) bilan qayta o'lchang — cho'qqi va ridge nuqtasi qanday o'zgaradi?
- GEMM'ni float64 da o'lchang (AI 2 marta kamayadi) va Roofline'dagi joyini ko'ring.
- Ishga tushirish xarajatini 0 qiling — qaysi yadrolar uchun bashorat keskin o'zgaradi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. Energiya samaradorligi (TOP/s/Vt, mJ/tasvir) va 3 yillik TCO

MAQSAD

ResNet-50 sinfidagi tasnif (~8,2 GOP/tasvir) va 5000 tasvir/s talab uchun har arxitekturada kerakli qurilmalar soni, umumiy quvvat, energiya (mJ/tasvir), CAPEX, elektr xarajati (PUE bilan) va egalik/bulut TCO hisoblanadi; natija bitta mln tasvir narxiga keltiriladi. Elektr narxi bo'yicha sezgirlik, 30 % yuklamada bulut va egalik solishtiruvi hamda chekka stsenariysi (120 tasvir/s, 30 Vt) ko'rib chiqiladi. Narx, quvvat va real foydalanish koeffitsientlari — taxminiy faraz, ular kod boshida o'zgartiriladi.

> **Kutilgan natija:** TOP/s/Vt: NPU ~10, GPU ~2,8, CPU ~0,08; mJ/tasvir NPU ~3,5, GPU ~10, CPU ~470; 24/7 da GPU egaligi bulutdan arzon, 30 % yuklamada bulut afzal; chekkada faqat NPU mos.

YO‘NALTIRUVCHI SAVOLLAR (SUQROT USULI)

Kod yozishdan oldin har bir savolga o‘zingiz qisqa javob yozing — javoblar sizni yechimga olib boradi. Ishoralar — qo‘llanma oxirida.

1. Bitta qurilma sekundiga nechta tasvirni qayta ishlay oladi? Buning uchun cho'qqi TOP/s, real foydalanish koeffitsienti va GOP/tasvir qanday birlashadi?

2. 5000 tasvir/s talabini qondirish uchun qurilmalar soni nega butun songa yuqoriga yaxlitlanadi? Umumiy quvvatga xost quvvati qanday kiradi?

3. mJ/tasvir qanday topiladi va nega u TOP/s/Vt bilan teskari bog'liq?

4. 3 yillik egalik TCO qaysi qismlardan iborat va PUE qayerga kiradi? Bulut TCO esa qanday hisoblanadi?

5. Narxni «sent / 1 mln tasvir» ga keltirish uchun 3 yilda jami nechta tasvir qayta ishlanadi?

6. Elektr narxi 4 marta oshsa, qaysi qurilmalarda TCO eng ko'p o'zgaradi? 30 % yuklamada bulut nega afzal bo'lib qoladi?

7. Chekka stsenariysida (120 tasvir/s, ≤ 30 Vt) qaysi qurilma mos kelishini kutasiz va nega TPU bu yerda umuman nomzod emas?

KARKAS — TODO JOYLARINI TO‘LDIRING (FAYL: S6_*.PY)

```
# 6-qadam. Energiya samaradorligi (TOP/s/Vt, J/inferens) va 3 yillik TCO taqqoslash
# Ssenariy: ResNet-50 sinfidagi tasnif (~8,2 GOP/tasvir), talab — 5000 tasvir/s, 24/7. Barcha
raqamlar TAXMINIY faraz.
import math
```

```
import pandas as pd
import matplotlib.pyplot as plt

pd.set_option("display.width", 170)
GOP_TASVIR = 8.2           # ResNet-50, bitta tasvir, GOP
TALAB = 5000               # tasvir/s
SOAT = 3 * 365 * 24        # 3 yil, soat
NARX_KVT = 0.10            # USD / kVt*soat (taxminiy)
PUE = 1.4                  # ma'lumot markazi: sovutish va boshqa yo'qotishlar koeffitsienti

# nom: cho'qqi INT8 TOP/s, real foydalanish, yuklamadagi quvvat Vt,
#      karta narxi USD, xost ulushi USD, xost quvvat ulushi Vt, bulut USD/soat (yo'q = None)
Q = {
    "CPU EPYC 9654":    (29.5, 0.30, 360, 11800, 4000, 150, None),
    "GPU H100 SXM":     (1979.0, 0.35, 700, 30000, 6000, 150, 3.00),
    "TPU v4 (bulut)":   (275.0, 0.50, 200, None, None, 0, 3.22),
    "NPU Hailo-8":      (26.0, 0.50, 2.5, 250, 300, 3, None),
    "Jetson AGX Orin":  (138.0, 0.30, 60, 2000, 0, 0, None),
    "FPGA Alveo U250":  (33.3, 0.60, 225, 9000, 3000, 100, None),
}
satr = []
for nom, (pk, u, vt, narx, xost, xvt, bulut) in Q.items():
    # TODO 1: bitta qurilmaning tasvir/s ini va talab uchun kerakli qurilmalar sonini toping
    # TODO 2: umumiy quvvat, 3 yillik energiya (PUE bilan, kVt*soat) va mJ/tasvir ni hisoblang
    # TODO 3: CAPEX, elektr xarajati, egalik va bulut TCO, sent / 1 mln tasvir ni hisoblang
(None larni hisobga oling)
    raise NotImplementedError("TODO 1-3: energiya va TCO")
df = pd.DataFrame(satr)
print(f"=== Energiya va TCO: talab {TALAB} tasvir/s, 3 yil, {NARX_KVT} $/kVt*soat, PUE {PUE}
(taxminiy) ===")
print(df.to_string(index=False))

print("\n=== Sezgirlik: egalik TCO (k$) elektr narxiga qarab ===")
# TODO 4: elektr narxi 0.05, 0.10, 0.20, 0.40 $ bo'lganda egalik TCO ni chiqaring
# TODO 5: 30 % yuklamada bulut (faqat ish vaqti) va egalik TCO ni solishtiring

print("\n=== Chekka stsenariysi: 120 tasvir/s, quvvat <= 30 Vt ===")
# TODO 6: har qurilma uchun 120 tasvir/s ga kerakli soni va quvvatni topib, 30 Vt byudjetga
mosligini ayting

print("\n=== TCO tahlili tugadi ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Energiya va TCO`, `=== Sezgirlik`, `=== Chekka stsenariysi`, `=== TCO tahlili tugadi ===`

TAJRIBA QILIB KO‘RING

- Real foydalanish koeffitsientlarini MLPerf Inference natijalari asosida yangilang (manbani hisobotda ko'rsating).
- Talabni 500 tasvir/s ga tushiring — qaysi arxitektura eng arzon bo'ladi?
- Jetson'ni 30 Vt rejimida (unumdorlik ~0,5x) hisoblang va chekka cheklovini qayta tekshiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

7 7-qadam. Yakuniy tanlov matritsasi: WSM, TOPSIS va vaznlar sezgirligi

MAQSAD

Oldingi qadamlar natijalari 6 mezonli jadvalga yig'iladi (unumdorlik, kechikish, energiya, TCO, ekotizim, moslashuvchanlik) va qat'iy cheklovlar (quvvat, internetsiz ishlash) qo'llanadi. Uch ssenariy (data-markaz, chekka kamera, moliya) uchun vaznli yig'indi (WSM) va TOPSIS reytinglari hisoblanadi; Dirichlet taqsimoti bilan vaznlar 2000 marta tebratilib, 1-o'rin barqarorligi baholanadi. Natija issiqlik xaritasi ko'rinishidagi tanlov matritsasi.

> **Kutilgan natija:** Data-markaz va moliyada GPU 1-o'rin (barqarorlik 80-98 %), chekkada Jetson/NPU — usullar kelishmasligi ko'rsatiladi; matritsa va grafik.

YO‘NALTIRUVCHI SAVOLLAR (SUQROT USULI)

Kod yozishdan oldin har bir savolga o‘zingiz qisqa javob yozing — javoblar sizni yechimga olib boradi. Ishoralar — qo‘llanma oxirida.

1. Mezonlar turli birliklarda (tasvir/s, ms, mJ, sent, ball) va turli yo'nalishda. Ularni bir-biriga qo'shishdan oldin nima qilish kerak?

2. Nega unumdorlik, energiya va TCO uchun log-shkala ishlatiladi? GPU ning 84000 tasvir/s qiymati chiziqli shkalada boshqa nomzodlarga nima qiladi?

3. Qat'iy cheklov (quvvat ≤ 60 Vt, oflayn) WSM ballaridan oldin yoki keyin qo'llanishi kerakmi? Nega?

4. TOPSIS ning g'oyasi qanday: «ideal» va «anti-ideal» nuqtalar qanday tuziladi va yakuniy ball qaysi nisbat?

5. Vaznlar sub'ektiv. Ularni Dirichlet bilan tebratish 1-o'rin barqarorligi haqida nima aytadi va konsentratsiya 50 nimani boshqaradi?

6. Natijani qanday tekshirasiz: data-markaz va moliyada GPU 1-o'rinda (barqarorlik 80–98 %) chiqdimi, chekkada Jetson/NPU va usullar kelishmasligi ko'rindimi?

KARKAS — TODO JOYLARINI TO‘LDIRING (FAYL: S7_*.PY)

```
# 7-qadam. Yakuniy tanlov matritsasi: ko'p mezonli qaror (WSM + TOPSIS) va vaznlar sezgirligi
(Monte-Karlo)
# Oldingi qadamlar natijalari (taxminiy) mezonlar jadvaliga yig'iladi; 3 ssenariy uchun turli
vaznlar bilan reyting.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
rng = np.random.default_rng(42)
pd.set_option("display.width", 160)

# --- Berilgan ma'lumotlar (o'zgartirmang) ---
MEZON = {"unumdorlik": +1, "kechikish": -1, "energiya": -1, "TCO": -1, "ekotizim": +1,
"moslashuvchanlik": +1}
JADVAL = pd.DataFrame({
    "unumdorlik":       [1080, 84000, 16800, 1590, 5050, 2440],
    "kechikish":        [6.0, 1.0, 1.5, 2.5, 2.0, 1.2],
    "energiya":         [473, 10.1, 11.9, 3.5, 11.9, 133],
    "TCO":              [18.7, 8.3, 17.9, 0.5, 0.5, 8.4],
    "ekotizim":         [5, 5, 4, 2, 4, 2],
    "moslashuvchanlik": [5, 4, 3, 2, 4, 5],
    "quvvat_vt":        [510, 850, 200, 5.5, 60, 325],
    "oflayn":           [1, 1, 0, 1, 1, 1],
}, index=["CPU", "GPU", "TPU", "NPU", "Jetson", "FPGA"])
print("=== Mezonlar jadvali (oldingi qadamlardan, taxminiy) ===")
print(JADVAL.to_string())

SSENARIY = {  # vaznlar yig'indisi 1; cheklov — lambda
    "Data-markaz: katta hajmli inferens": ({"unumdorlik": .30, "kechikish": .10, "energiya":
.15, "TCO": .25, "ekotizim": .15, "moslashuvchanlik": .05}, lambda r: True),
    "Chekka kamera: <= 60 Vt, internetsiz":  ({"unumdorlik": .10, "kechikish": .20, "energiya":
.35, "TCO": .15, "ekotizim": .10, "moslashuvchanlik": .10}, lambda r: r.quvvat_vt <= 60 and
r.oflayn == 1),
    "Moliya: past kechikish, tez o'zgaruvchi model": ({"unumdorlik": .10, "kechikish": .35,
"energiya": .05, "TCO": .10, "ekotizim": .15, "moslashuvchanlik": .25}, lambda r: r.oflayn ==
1),
}


def normal(df):
    """Min-max normallash: 1 — eng yaxshi, 0 — eng yomon (yo'nalish hisobga olinadi)."""
    # TODO 1: har mezon uchun (unumdorlik/energiya/TCO — log10 shkalada) [0, 1] ga keltiring,
-1 yo'nalishni teskari qiling
    raise NotImplementedError("TODO 1: normallash")


def topsis(df, w):
    """TOPSIS: ideal va anti-ideal yechimgacha masofa nisbati."""
    # TODO 2: vektor normallash, vaznlash, ideal/anti-ideal nuqta, masofalar va dm / (dp + dm)
    raise NotImplementedError("TODO 2: TOPSIS")


issiq = {}
for nom, (w, cheklov) in SSENARIY.items():
    # TODO 3: cheklov bo'yicha nomzodlarni filtrlang, WSM va TOPSIS ballarini hisoblang
    # TODO 4: Dirichlet (W0*50) bilan 2000 marta vaznlarni tebratib, har nomzodning 1-o'rin
ulushini (%) toping
    # TODO 5: natijani WSM bo'yicha tartiblab chiqaring, WSM va TOPSIS tavsiyalari kelishganini
tekshiring
    print(f"\n=== Ssenariy: {nom} ===")
    raise NotImplementedError("TODO 3-5: ssenariy reytingi")

print("\n=== Yakuniy tanlov matritsasi (WSM, '—' = cheklov bo'yicha mos emas) ===")
# TODO 6: issiq dan DataFrame tuzib chiqaring va imshow bilan issiqlik xaritasini chizing
print("\n=== Tanlov matritsasi tayyor ===")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Mezonlar jadvali`, `=== Ssenariy:`, `=== Yakuniy tanlov matritsasi`, `=== Tanlov matritsasi tayyor ===`

TAJRIBA QILIB KO‘RING

- Data-markaz ssenariysiga 'quvvat_vt >= 100' (faqat server qurilmalari) cheklovini qo'shing va reytingni qayta ko'ring.
- Ekotizim va moslashuvchanlik ekspert baholarini o'zgartiring — tavsiya qachon o'zgaradi?
- To'rtinchi ssenariy (LLM dekod xizmati) qo'shing: xotira o'tkazuvchanligi mezoni bilan.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Ilova. Ishoralar

Avval savolga o‘zingiz javob berishga harakat qiling; ishorani faqat qiynalganda o‘qing. To‘liq namuna yechim saytdagi virtual laboratoriyada bosqichni bajarganingizdan keyin ochiladi.

1. 1-qadam. Arxitekturalar pasporti va Roofline modeli

1. Biri — hisoblash cho'qqisi (TOP/s), ikkinchisi — xotira o'tkazuvchanligi bilan AI ko'paytmasi. Birliklarga e'tibor bering: TOP/s = 1e12, GB/s = 1e9.
2. Ridge = cho'qqi / o'tkazuvchanlik (op/bayt). Hailo-8 da BW atigi 4 GB/s — shuning uchun ridge juda katta chiqadi.

```
df["ridge_int8"] = df.int8_tops * 1e12 / (df.bw_gbs * 1e9)
```

3. Har bir chiqish elementi uchun K ta ko'paytirish va K ta qo'shish. Ideal holda A, B va C matritsalar bir martadan o'qiladi/yoziladi; INT8 da element = 1 bayt.
4. Oddiy conv: $2 \cdot H \cdot W \cdot Ci \cdot Co \cdot k^2$ amal. Depthwise da har kanal alohida (Ci = Co = C, kanallar aralashmaydi) — shuning uchun AI keskin tushadi.
5. P = min(P_cho'qqi, AI·BW). Agar cho'qqi AI·BW dan kichik yoki teng bo'lsa — hisoblash chegarasi. BW ni GB/s dan TB/s ga o'tkazish uchun 1e3 ga bo'ling.

```
p = min(q.int8_tops, ai * q.bw_gbs / 1e3)
```

6. 7B parametr INT8 da ≈ 7 GB. token/s ≈ BW / model hajmi; hisoblash cho'qqisi bu yerda deyarli rol o'ynamaydi.
7. GEMM 4096^3: $2 \cdot 4096^3 / (3 \cdot 4096^2) \approx 2731$. GEMV: $2 \cdot 4096^2 / (4096^2 + ...) \approx 2$. Agar shunday bo'lmasa — birliklarni (1e12/1e9/1e3) tekshiring.

Bajargach o‘ylang: Namunada chegara turi P ni hisoblash bilan bir xil ifodadan aniqlanadi — sizda ham shundaymi? FP16 (b=2) bo'lsa qaysi yuklamalar chegara turini o'zgartirishi mumkinligini taxmin qiling.

2. 2-qadam. TPU sistolik massivi: takt darajasidagi simulyator va MXU foydalanish modeli

1. Natija (acc) har PE ichida to'planadi; A qiymatlari o'ngga, B qiymatlari pastga siljiydi.
2. i-satr t − i taktda boshlanadi (skew). Oxirgi ko'paytma t = (K−1)+(R−1)+(C−1) da keladi, demak taktlar soni shundan bitta ko'p.
3. Chap ustun bo'shaydi va unga yangi qiymat kiradi; qolganlari bir ustun o'ngga ko'chadi. Ustma-ust tushgan kesmalar nusxasiz buzilishi mumkin.

```
a_reg[:, 1:] = a_reg[:, :-1].copy()
```

4. Chegaradan tashqarida 0 kiritiladi. A va B da nol yo'q (1..8), shuning uchun (a_reg != 0) & (b_reg != 0) — faol MAC belgisi.

5. Plitkalar = ceil(K/128)·ceil(N/128); har plitka M + 128 + 128 − 2 takt (+128 yuklash, agar ikki bufer bo'lmasa). Foydalanish = M·K·N / (128·128·jami_takt).
6. MXU modelida BERT 4096 token ~94 %, GEMV M=1 ~0,4 %, N=130 ~45 % bo'lishi kerak. GEMV nega shunchalik past — M=1 da fill/drain qancha ulush oladi?

Bajargach o‘ylang: Namunada skew kirish indeksidagi t − i orqali amalga oshirilgan — siz uni boshqacha (masalan, A ni oldindan siljitib) qildingizmi? N=130 holatida padding isrofini qanday kamaytirish mumkin?

3. 3-qadam. FPGA: quvur (pipeline), II va fixed-point aniqlik simulyatsiyasi

1. Boshlanish = i·II, tugash = boshlanish + bosqichlar soni. Jami takt — oxirgi kirishning tugash vaqti.
2. Kutilgan qiymat: 8 kirish 11 taktda (to'ldirish 3 takt + 8). II=2, II=4 da o'tkazuvchanlik taxminan II marta tushadi.
3. 1 MAC = 2 amal. INT8 packing'da bitta DSP 2 MAC bajaradi, FP32 esa ~3 DSP talab qiladi. INT8 packing uchun ~12 TOP/s kutiladi.

```
tops = DSP * ulush * mac_dsp * 2 * F / II / 1e12
```

4. x·2^n ni yaxlitlang (np.round), keyin ishorali jami-bit diapazoniga np.clip bilan to'yintiring. To'yinganlar sonini ham hisoblang.

```
q = np.round(x * (1 << kasr))
```

5. Ko'paytmada kasr bitlar qo'shiladi: kasr_a + kasr_w. ReLU dan keyin >> kasr_w qilib yana Q kasr_a ga qaytasiz — FPGA'da siljitish deyarli bepul.
6. Yig'indi to'yinib, chiqishlar buziladi. SQNR = 10·log10(Σ etalon² / Σ (etalon − kvant)²), bu yerda kvant chiqish 2^(kasr_a+kasr_w) ga bo'linadi.
7. Q0.7 da butun qism yo'q — |w| ≥ 1 bo'lgan og'irliklar to'yinadi. Og'irliklar diapazonini chiqarib ko'ring.

Bajargach o‘ylang: Namunadagi fixed_mlp bilan solishtiring: bias va siljitishni siz ham xuddi shu tartibda qildingizmi? Q0.7 o'rniga Q1.6 tanlash qaysi muammoni hal qiladi?

4. 4-qadam. NPU uchun INT8 kvantlash: aniqlik, hajm, kechikish va grafik bo'linishi

1. Faollashtirishlar ham kvantlanadi; ularning diapazoni (min/max) kuzatuvchilar (observer) orqali kalibrlash namunalaridan yig'iladi.
2. state_dict ni io.BytesIO ga torch.save qiling. Bias, masshtab va zero_point'lar qo'shimcha joy oladi.

```
b = io.BytesIO(); torch.save(model.state_dict(), b)
```

3. fuse_modules → prepare → kalibrlash (bir marta forward) → convert. Birlashtirilgan qatlam oraliq tenzorni alohida kvantlamaydi.

```
q = tq.fuse_modules(q, [["c1", "r1"], ...])
```

4. Per-channel har chiqish kanaliga alohida masshtab beradi: PerChannelMinMaxObserver + torch.per_channel_symmetric. Faollashtirish uchun quint8 MinMaxObserver.
5. s = max|w| / 127; wq = clip(round(w/s), −127, 127)·s; SQNR = 10·log10(Σw² / Σ(w−wq)²). ~40–47 dB kutiladi.
6. Har operator: NPU yoki CPU hisoblash vaqti. Joriy joy (CPU/NPU) o'zgarsa — chiqish tenzori MB / PCIe GB/s ms qo'shiladi. Kirish CPU xotirasida turadi.
7. Batch=1 da INT8 sekinroq chiqsa — bu xato emas: kichik modelda quant/dequant xarajati ustun keladi.

Bajargach o‘ylang: Namunadagi kvantla() bilan solishtiring: kalibrlash torch.no_grad() ichida bajarilganmi? 8 va 256 namunali kalibrlash orasida farq kichik chiqqani nimani anglatadi?

5. 5-qadam. Real CPU'da GEMM va konvolyutsiya o'lchovlari, boshqa arxitekturalarga ekstrapolyatsiya

1. Amaliy cho'qqi — eng tez ishlagan GEMM ning GFLOP/s qiymati; amaliy BW — STREAM triad. Ikkalasi birgalikda «o'lchangan Roofline» ni beradi.
2. 1–2 marta isitish (kesh, xotira ajratish), keyin bir necha o'lchov va median — tasodifiy uzilishlarga chidamli.

```
t0 = time.perf_counter(); f(); t.append(time.perf_counter() - t0)
```

3. b va c o'qiladi, a yoziladi: 3·n·4 bayt. out= parametri yangi xotira ajratmaslik uchun kerak.

```
torch.add(b, c, alpha=1.5, out=a)
```

4. ops = 2·batch·H·W·Co·(Ci/g)·k²; bayt = 4·(kirish + og'irlik + chiqish elementlari). Depthwise da g = Ci, shuning uchun AI juda kichik.
5. Kichik matritsalar L2/L3 keshga sig'adi va RAM triad BW dan tezroq o'qiladi. Depthwise samaradorligi < 10 % kutiladi.
6. FP32 baytlari 4 ga bo'linib, FP16 da 2 ga, INT8 da 1 ga ko'paytiriladi. Kichik yadrolar tezlatkichda 5–10 us «pol» ga uriladi.
7. 1e9 ga bo'lish, float32 = 4 bayt, GEMM da 2·N³ amal — eng ko'p uchraydigan xatolar shu yerda.

Bajargach o‘ylang: Namunadagi e ni 5..100 % ga cheklash qanday maqsadga xizmat qiladi? Sizning ekstrapolyatsiyangiz PCIe nusxalashni hisobga oladimi va bu natijani qanday o'zgartirardi?

6. 6-qadam. Energiya samaradorligi (TOP/s/Vt, mJ/tasvir) va 3 yillik TCO

1. Real unumdorlik = cho'qqi · u (TOP/s); 1 TOP = 1000 GOP. Uni 8,2 GOP/tasvir ga bo'ling.

```
ips = pk * u * 1e3 / GOP_TASVIR
```

2. math.ceil(TALAB / ips). Har dona uchun karta + xost ulushi Vt qo'shiladi.
3. Energiya/tasvir = quvvat (Vt = J/s) / tasvir/s. 1e3 ga ko'paytirib mJ ga o'ting. Kutilgan: NPU ~3,5, GPU ~10, CPU ~470 mJ.

4. Egalik = CAPEX (karta + xost) · n + elektr (P · soat / 1000 · PUE · narx). Bulut = n · USD/soat · soat. Narxi None bo'lgan variantni hisoblamang.
5. TALAB · SOAT · 3600 tasvir; eng arzon TCO (egalik yoki bulut) ni shu songa bo'lib, mln va sentga o'tkazing.
6. Sezgirlik: CAPEX o'zgarmaydi, elektr qismi narxga proporsional. Bulut faqat ishlagan soat uchun to'lanadi, egalik CAPEX esa baribir to'lanadi.
7. Kutilgan natija: faqat NPU mos. TPU v4 faqat bulutda mavjud (narx None).

Bajargach o‘ylang: Namunadagi tco = min(egalik, bulut) yondashuvini o'zingiznikiga solishtiring. Qaysi faraz (foydalanish koeffitsienti, PUE yoki narx) natijaga eng kuchli ta'sir qiladi deb o'ylaysiz?

7. 7-qadam. Yakuniy tanlov matritsasi: WSM, TOPSIS va vaznlar sezgirligi

1. Har mezonni [0, 1] ga keltiring: 1 — eng yaxshi. «Kami yaxshi» (−1) mezonlar uchun 1 − z oling.
2. Diapazon bir necha tartibga cho'zilgan; chiziqli min-max da qolganlar hammasi ~0 ga yopishib qoladi.

```
x = np.log10(df[m].astype(float))
```

3. Avval filtr: mos kelmaydigan nomzod normallash diapazoniga ham ta'sir qilmasligi kerak. JADVAL.apply(cheklov, axis=1) mantiqiy niqob beradi.
4. Ustunlarni $\sqrt{\Sigma x^2}$ ga bo'lib, vaznga ko'paytiring; ideal = yaxshi yo'nalishdagi ekstremum. Ball = $d^- / (d^+ + d^-)$.
5. rng.dirichlet(W0 * 50) — o'rtacha W0 atrofida, yig'indisi 1 bo'lgan vazn. Konsentratsiya qancha katta bo'lsa, tebranish shuncha kichik.

```
wv = rng.dirichlet(W0 * 50)
```

6. Chekkada faqat 2 nomzod qoladi — min-max ularni 0/1 ga aylantiradi, shuning uchun WSM va TOPSIS turli javob berishi mumkin.

Bajargach o‘ylang: Namunadagi WSM va TOPSIS natijalarini o'zingiznikiga solishtiring. Qaysi usul nomzodlar kam bo'lganda ishonchliroq va nega?

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**Pasport jadvalini tekshirish** 1-qadam jadvalidagi har qurilma uchun cho'qqi INT8/FP16 qiymati, xotira o'tkazuvchanligi va quvvatni ishlab chiqaruvchining rasmiy ma'lumot varag'i (datasheet) bilan solishtiring; farq bo'lsa tuzating va manbalar ro'yxatini (havola yoki hujjat nomi, sana) jadval ostida bering. Ridge nuqtalarini qayta hisoblang.	oson	5
2	**O'z yuklamangizning AI si** 1-qadamga o'zingiz tanlagan modelning 3 ta qatlamini (masalan, YOLO conv, Transformer attention QK^T, embedding qidiruv) qo'shing: amallar va baytlar formulasini yozing, AI ni hisoblang va har arxitekturada chegara turini (H/X) jadvalda ko'rsating.	oson	5
3	**Sistolik massiv to'g'riligi va to'lqin fronti** 2-qadam simulyatorini 12x12 massiv va K=12, 48, 192 uchun ishga tushiring, taktlar formulasini tasdiqlang va t=5, 11, 17 dagi faol PE xaritalarini chiqaring. PE foydalanishning K ga bog'liqligi grafigini topshiring.	oson	5
4	**MXU uchun padding strategiyasi** 2-qadam modelida N = 100...300 (qadam 10) uchun MXU foydalanishini hisoblang, 128 ga karrali bo'lmagan o'lchamlardagi 'arra tishi' grafigini chizing. Toq o'lchamli qatlam uchun 3 strategiyani (padding, o'lchamni qisqartirish, ikki qatlamni birlashtirish) samarali TFLOP/s bo'yicha solishtiring.	o'rta	10
5	**FPGA fixed-point dizaynini tanlash** 3-qadamdagi MLP uchun har qatlamga alohida (og'irlik, faollashtirish, akkumulyator) bit kengligini tanlang: aniqlik pasayishi <= 0,5 p.p. bo'lgan eng tejamkor konfiguratsiyani toping. DSP soni va o'tkazuvchanlikni (300 MHz, II=1) modelda baholang va jadvalda bering.	o'rta	10
6	**Kalibrlash va observer tadqiqoti** 4-qadamda MinMax, MovingAverageMinMax va Histogram observerlarini 4, 16, 64, 256 namunali kalibrlash bilan solishtiring (aniqlik, kalibrlash vaqti). Natijani 4x3 jadvalda va qisqa xulosada bering: NPU kompilyatoriga qaysi kalibrlash to'plamini berardingiz?	o'rta	10
7	**Sandbox CPU Roofline'ini aniqlashtirish** 5-qadamda triad o'lchovini 1, 4, 16, 64 MB massivlar bilan takrorlang (kesh ierarxiyasi ko'rinsin) va GEMM'ni 1 va 2 ip bilan o'lchang. Kesh darajalari uchun alohida 'shift' chiziqli Roofline grafigini chizing va GEMV ning 100 % dan yuqori samaradorligini tushuntiring.	o'rta	10
8	**TCO modeliga o'z mintaqangiz narxlari** 6-qadam modelini O'zbekiston sharoitiga moslang: elektr narxi (amaldagi tarif, manba bilan), import bojxona xarajatlari, sovutish (PUE) va xizmat ko'rsatish (yillik 10 % CAPEX) qo'shing. 3 ta talab darajasi (500, 5000, 50000 tasvir/s) uchun TCO va 1 mln tasvir narxini hisoblang; natija qaysi arxitekturani tanlashga ta'sir qilishini tahlil qiling. Barcha farazlarni taxminiy deb belgilang.	qiyin	15

9	**Ekstrapolyatsiyani tekshirish** 5-qadam usuli bilan ONNX Runtime'da kichik CNN (yoki digits MLP) inferensini sandbox CPU'da o'lchang, so'ng uni qatlam bo'yicha Roofline modelida jamlab boshqa arxitekturalar uchun bashorat qiling. Bashoratni ochiq manbadagi (MLPerf Inference yoki ishlab chiqaruvchi benchmark) mos model natijasi bilan solishtiring va xato manbalarini (ishga tushirish xarajati, fusion, kesh) tahlil qiling.	qiyin	15
10	**To'liq qiyosiy hisobot va tanlov matritsasi** O'z AI ilovangiz (masalan, shahar kameralari, tibbiy tasvir, chatbot) uchun 1-7 qadamlarni moslab, CPU/GPU/TPU/NPU/FPGA bo'yicha yakuniy hisobot tayyorlang: pasport, Roofline, kvantlash ta'siri, energiya va TCO, kamida 3 ssenariyli tanlov matritsasi va vaznlar sezgirligi. Tavsiyani raqamlar bilan asoslang (kamida 400 so'z), barcha taxminiy qiymatlarni alohida belgilang.	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. Tibbiyot: KT tasvirlarida o'smani segmentlash serveri oson

Soha: Tibbiyot (radiologiya)

Viloyat diagnostika markazi KT kesimlarini U-Net bilan segmentlovchi server sotib olmoqchi. Kuniga ~3000 kesim, har biri 512x512, natija 2 daqiqa ichida kerak. Bemor ma'lumoti binodan chiqmasligi sababli bulut faqat taqqoslash uchun ko'riladi. Siz CPU, GPU va FPGA ni Roofline, energiya va TCO bo'yicha solishtirib, tavsiya berasiz.

Yuklama/model	U-Net conv3x3 512x512x64->64 va 1x1 qatlam
Talab	3000 kesim/kun, javob <= 120 s
Cheklov	bulutsiz, 1 kVt gacha
Solishtiriladi	CPU vs GPU vs FPGA
Fixed-point format	Q2.5 / Q4.11
Narx farazi	elektr 0,08 $/kVt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (U-Net conv3x3 512x512x64->64 va 1x1 qatlam) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 3- va 4-qadamlar: fixed-point (Q2.5 / Q4.11) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
4. 6-qadam: talab (3000 kesim/kun, javob <= 120 s) va cheklov (bulutsiz, 1 kVt gacha) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (elektr 0,08 $/kVt*soat); sezgirlik tahlili.
5. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

2-variant. Agro: dron tasvirlarida begona o'tni aniqlash oson

Soha: Qishloq xo'jaligi

Paxta dalasida dron 30 kadr/s 1280x720 tasvir oladi, begona o'tlar real vaqtda aniqlanib purkagichga signal beriladi. Dron batareyasi chekli: hisoblash moduli 15 Vt dan oshmasligi kerak. Hailo-8 kabi NPU va Jetson sinfidagi SoC bilan bort CPU ni solishtirasiz.

Yuklama/model	YOLO uslubidagi depthwise conv + 1x1
Talab	30 kadr/s, kechikish <= 50 ms
Cheklov	<= 15 Vt, internetsiz
Solishtiriladi	CPU vs NPU vs Jetson
Fixed-point format	Q1.6 / Q3.4
Narx farazi	batareya 100 Vt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs NPU vs Jetson + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (YOLO uslubidagi depthwise conv + 1x1) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 3- va 4-qadamlar: fixed-point (Q1.6 / Q3.4) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
4. 6-qadam: talab (30 kadr/s, kechikish <= 50 ms) va cheklov (<= 15 Vt, internetsiz) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (batareya 100 Vt*soat); sezgirlik tahlili.
5. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

3-variant. Transport: avtomobil raqamini tanish (ANPR) shlyuzi oson

Soha: Transport

Shahar chorrahalarida 16 ta kamera, har biridan 10 kadr/s; raqam aniqlash va OCR mahalliy shlyuzda bajariladi. Shlyuz shkafda turadi, sovutish passiv, 60 Vt gacha. Arxitekturalarni kadr/s/Vt va 5 yillik TCO bo'yicha baholaysiz.

Yuklama/model	conv3x3 + kichik LSTM/GEMM (OCR)
Talab	160 kadr/s
Cheklov	<= 60 Vt, passiv sovutish
Solishtiriladi	CPU vs Jetson vs FPGA
Fixed-point format	Q3.4
Narx farazi	5 yil, 0,10 $/kVt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs Jetson vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (conv3x3 + kichik LSTM/GEMM (OCR)) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 3- va 4-qadamlar: fixed-point (Q3.4) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
4. 6-qadam: talab (160 kadr/s) va cheklov (<= 60 Vt, passiv sovutish) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (5 yil, 0,10 $/kVt*soat); sezgirlik tahlili.
5. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

4-variant. Telekom: tarmoq trafigidagi anomaliyalarni aniqlash oson

Soha: Telekom

Operator yadro tarmog'ida har soniyada 2 mln paket metadatasi avtoenkoder bilan tekshiriladi. Kechikish 1 ms dan oshsa paket navbati to'lib ketadi. FPGA (SmartNIC) bilan GPU va CPU ni GEMV-og'ir yuklamada solishtirasiz.

Yuklama/model	MLP avtoenkoder 64-32-8-32-64 (GEMV, batch kichik)
Talab	2 mln namuna/s, p99 <= 1 ms
Cheklov	server shkafi, 300 Vt
Solishtiriladi	CPU vs GPU vs FPGA
Fixed-point format	Q1.14 (16 bit)
Narx farazi	elektr 0,12 $/kVt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (MLP avtoenkoder 64-32-8-32-64 (GEMV, batch kichik)) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 3- va 4-qadamlar: fixed-point (Q1.14 (16 bit)) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
4. 6-qadam: talab (2 mln namuna/s, p99 <= 1 ms) va cheklov (server shkafi, 300 Vt) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (elektr 0,12 $/kVt*soat); sezgirlik tahlili.
5. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

5-variant. Energetika: quyosh paneli nuqsonlarini termal tasvirda topish oson

Soha: Energetika

Quyosh elektr stansiyasida 40 ming panel, dron termal kamerasi bilan haftasiga bir marta suratga olinadi (~200 ming tasvir). Ishlov kechasi paketli bajariladi, kechikish muhim emas, narx muhim. Bulut TPU, egalikdagi GPU va CPU klasterini solishtirasiz.

Yuklama/model	ResNet-18 sinfidagi tasnif (1,8 GMAC)
Talab	200 ming tasvir 8 soatda
Cheklov	kechikish muhim emas
Solishtiriladi	CPU vs GPU vs TPU
Fixed-point format	BF16
Narx farazi	bulut TPU soatbay

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs TPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (ResNet-18 sinfidagi tasnif (1,8 GMAC)) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 3- va 4-qadamlar: fixed-point (BF16) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
4. 6-qadam: talab (200 ming tasvir 8 soatda) va cheklov (kechikish muhim emas) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (bulut TPU soatbay); sezgirlik tahlili.
5. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

6-variant. Moliya: kartadagi firibgarlikni real vaqtda baholash oson

Soha: Moliya (bank)

Bank protsessing markazida har tranzaksiya 5 ms ichida gradient boosting + kichik MLP bilan baholanishi kerak; cho'qqida 8000 tranzaksiya/s. Ishonchlilik uchun ikki ma'lumot markazi. CPU, GPU va FPGA ni p99 kechikish va TCO bo'yicha solishtirasiz.

Yuklama/model	MLP 256-128-1 + daraxtlar (batch 1)
Talab	8000 tranz./s, p99 <= 5 ms
Cheklov	2 DC, zaxira 2N
Solishtiriladi	CPU vs GPU vs FPGA
Fixed-point format	Q3.12
Narx farazi	3 yil, 2 ta DC

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (MLP 256-128-1 + daraxtlar (batch 1)) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 3- va 4-qadamlar: fixed-point (Q3.12) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
4. 6-qadam: talab (8000 tranz./s, p99 <= 5 ms) va cheklov (2 DC, zaxira 2N) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (3 yil, 2 ta DC); sezgirlik tahlili.
5. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

7-variant. Ta'lim: imtihon videonazorati (proktoring) xizmati oson

Soha: Ta'lim

Universitet onlayn imtihonida 2000 talaba bir vaqtda, har biridan 2 kadr/s yuz va ko'z harakati tahlili qilinadi. Imtihonlar yiliga 30 kun, qolgan vaqt yuklama past. Egalikdagi GPU serveri va bulut GPU/TPU ni yuklama koeffitsienti bo'yicha solishtirasiz.

Yuklama/model	MobileNetV2 (0,3 GMAC) + yuz nuqtalari
Talab	4000 kadr/s, 30 kun/yil
Cheklov	mavsumiy yuklama
Solishtiriladi	GPU egalik vs bulut GPU vs TPU
Fixed-point format	INT8
Narx farazi	yuklama koeffitsienti 8 %

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (GPU egalik vs bulut GPU vs TPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (MobileNetV2 (0,3 GMAC) + yuz nuqtalari) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 3- va 4-qadamlar: fixed-point (INT8) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
4. 6-qadam: talab (4000 kadr/s, 30 kun/yil) va cheklov (mavsumiy yuklama) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (yuklama koeffitsienti 8 %); sezgirlik tahlili.
5. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

8-variant. Sanoat: konveyerda metall quyma nuqsonini aniqlash o'rta

Soha: Sanoat (metallurgiya)

Quyish sexida konveyer 2 m/s tezlikda, 4 kamera 120 kadr/s. Nuqsonli detal 20 ms ichida ajratilishi kerak, sex harorati yuqori va chang ko'p. FPGA (deterministik kechikish) va Jetson ni CPU bilan solishtirasiz.

Yuklama/model	conv3x3 256x256x32->32, 5 qatlam
Talab	480 kadr/s, kechikish <= 20 ms (qat'iy)
Cheklov	IP65 shkaf, 100 Vt
Solishtiriladi	CPU vs Jetson vs FPGA
MXU/sistolik holat	N=32
Fixed-point format	Q2.5 / Q2.13
Narx farazi	elektr 0,07 $/kVt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs Jetson vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (conv3x3 256x256x32->32, 5 qatlam) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (N=32) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (Q2.5 / Q2.13) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (480 kadr/s, kechikish <= 20 ms (qat'iy)) va cheklov (IP65 shkaf, 100 Vt) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (elektr 0,07 $/kVt*soat); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

9-variant. Xavfsizlik: aeroportda bagaj rentgen tasvirini skrining o'rta

Soha: Xavfsizlik

Aeroport bagaj liniyasida soatiga 1800 sumka, har biri 4 proyeksiyali rentgen tasvir. Taqiqlangan buyumni aniqlash detektori 1 s ichida operatorga ko'rsatishi kerak. GPU serveri, FPGA va CPU ni ishonchlilik va TCO bo'yicha taqqoslaysiz.

Yuklama/model	Faster R-CNN sinfidagi detektor (~90 GMAC)
Talab	2 tasvir/s, javob <= 1 s
Cheklov	ishonchlilik 99,99 %
Solishtiriladi	CPU vs GPU vs FPGA
MXU/sistolik holat	M=4
Fixed-point format	Q4.11
Narx farazi	7 yil xizmat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (Faster R-CNN sinfidagi detektor (~90 GMAC)) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (M=4) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (Q4.11) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (2 tasvir/s, javob <= 1 s) va cheklov (ishonchlilik 99,99 %) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (7 yil xizmat); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

10-variant. Chakana savdo: do'kon javonlarini monitoring qilish o'rta

Soha: Chakana savdo

Supermarketlar tarmog'i 120 do'konda javonlardagi bo'sh joylarni har 5 daqiqada tekshiradi; har do'konda 20 kamera. Variant: har do'konda chekka qurilma yoki markaziy GPU serveri. Energiya va trafik narxi bilan 3 yillik TCO hisoblaysiz.

Yuklama/model	EfficientDet-D0 (2,5 GMAC)
Talab	120 do'kon x 20 kamera / 5 daq
Cheklov	do'konda 20 Vt gacha
Solishtiriladi	NPU (har do'kon) vs markaziy GPU vs CPU
MXU/sistolik holat	M=8
Fixed-point format	INT8 per-channel
Narx farazi	trafik 0,05 $/GB

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (NPU (har do'kon) vs markaziy GPU vs CPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (EfficientDet-D0 (2,5 GMAC)) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (M=8) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (INT8 per-channel) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (120 do'kon x 20 kamera / 5 daq) va cheklov (do'konda 20 Vt gacha) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (trafik 0,05 $/GB); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

11-variant. Logistika: ombor robotlari uchun yo'l rejalashtirish o'rta

Soha: Logistika

Avtomatlashtirilgan omborda 150 robot, har biri 10 Gs chastotada lidar nuqtalar bulutini kichik PointNet bilan tahlil qiladi. Hisoblash robot bortida (Jetson) yoki markaziy serverda (Wi-Fi kechikishi 15 ms) bo'lishi mumkin. Variantlarni kechikish va energiya bo'yicha solishtirasiz.

Yuklama/model	PointNet: GEMM 1024x64->128 va max-pool
Talab	150 robot x 10 Gs
Cheklov	bort batareyasi, 25 Vt
Solishtiriladi	Jetson (bort) vs GPU (markaz) vs CPU
MXU/sistolik holat	M=1024
Fixed-point format	FP16
Narx farazi	Wi-Fi RTT 15 ms

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (Jetson (bort) vs GPU (markaz) vs CPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (PointNet: GEMM 1024x64->128 va max-pool) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (M=1024) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (FP16) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (150 robot x 10 Gs) va cheklov (bort batareyasi, 25 Vt) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (Wi-Fi RTT 15 ms); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

12-variant. Shahar: aqlli svetofor uchun transport oqimini bashorat qilish o'rta

Soha: Shahar infratuzilmasi

Shahar 300 chorrahasida har 10 s da GNN/LSTM bilan keyingi 5 daqiqa oqimi bashorat qilinadi. Hisob markaziy data-markazda, kechikish 2 s gacha. GEMV-og'ir LSTM yuklamada xotira o'tkazuvchanligi hal qiluvchi ekanini Roofline bilan ko'rsatasiz.

Yuklama/model	LSTM 512 yashirin, 300 ketma-ketlik
Talab	30 bashorat/s, kechikish <= 2 s
Cheklov	data-markaz
Solishtiriladi	CPU vs GPU vs TPU
MXU/sistolik holat	M=300
Fixed-point format	BF16
Narx farazi	PUE 1,6

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs TPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (LSTM 512 yashirin, 300 ketma-ketlik) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (M=300) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (BF16) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (30 bashorat/s, kechikish <= 2 s) va cheklov (data-markaz) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (PUE 1,6); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

13-variant. Ekologiya: havo sifati sensorlarida chekka tahlil o'rta

Soha: Ekologiya

Shahar bo'ylab 500 ta havo sifati stansiyasi quyosh paneli bilan ishlaydi (o'rtacha 5 Vt). Har stansiya har daqiqada 1D-CNN bilan ifloslanish manbasini tasniflaydi. Mikro-NPU, kichik CPU (Raspberry Pi sinfi) va bulutga yuborishni energiya bo'yicha solishtirasiz.

Yuklama/model	1D-CNN 6 qatlam, 1440 nuqta
Talab	1 tasnif/daq/stansiya
Cheklov	<= 5 Vt, quyosh
Solishtiriladi	CPU (RPi) vs NPU vs bulut
MXU/sistolik holat	N=24
Fixed-point format	Q1.6 (8 bit)
Narx farazi	LTE trafik 2 $/GB

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU (RPi) vs NPU vs bulut + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (1D-CNN 6 qatlam, 1440 nuqta) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (N=24) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (Q1.6 (8 bit)) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (1 tasnif/daq/stansiya) va cheklov (<= 5 Vt, quyosh) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (LTE trafik 2 $/GB); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

14-variant. Sport: futbol o'yinida o'yinchilarni kuzatish o'rta

Soha: Sport

Stadionda 8 kamera 50 kadr/s 4K oqim beradi, o'yinchi va to'p koordinatalari 100 ms kechikish bilan efirga chiqariladi. O'yinlar haftasiga 2 marta, 2 soatdan. Mobil GPU serveri va bulut GPU ni foydalanish koeffitsienti bilan baholaysiz.

Yuklama/model	YOLOv8-m sinfidagi detektor (~40 GMAC)
Talab	400 kadr/s, <= 100 ms
Cheklov	mobil stend, 2 kVt
Solishtiriladi	GPU egalik vs bulut GPU vs FPGA
MXU/sistolik holat	M=8
Fixed-point format	FP16 / INT8
Narx farazi	haftada 4 soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (GPU egalik vs bulut GPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (YOLOv8-m sinfidagi detektor (~40 GMAC)) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (M=8) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (FP16 / INT8) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (400 kadr/s, <= 100 ms) va cheklov (mobil stend, 2 kVt) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (haftada 4 soat); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

15-variant. Media: video kontentni avtomatik subtitrlash o'rta

Soha: Media

Teleradiokompaniya arxivida 20 ming soat video; Whisper sinfidagi model bilan o'zbek tilida subtitr yaratiladi. Kodlovchi qism hisobga, dekoder xotiraga og'ir. Bulut TPU, GPU va CPU klasterida arxivni 30 kunda qayta ishlash narxini solishtirasiz.

Yuklama/model	Transformer kodlovchi (GEMM 1500x512x2048) + dekoder GEMV
Talab	20 ming soat / 30 kun
Cheklov	faqat narx va muddat
Solishtiriladi	CPU vs GPU vs TPU
MXU/sistolik holat	M=1500 (kodlovchi), M=1 (dekoder)
Fixed-point format	BF16
Narx farazi	bulut soatbay

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs TPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (Transformer kodlovchi (GEMM 1500x512x2048) + dekoder GEMV) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (M=1500 (kodlovchi), M=1 (dekoder)) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (BF16) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (20 ming soat / 30 kun) va cheklov (faqat narx va muddat) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (bulut soatbay); sezgirlik tahlili.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

16-variant. Tibbiyot: EKG signalidan aritmiyani taqiladigan qurilmada aniqlash

qiyin

Soha: Tibbiyot (kardiologiya)

Taqiladigan EKG qurilmasi 250 Gs signalni 1D-CNN bilan har 10 s da tahlil qiladi, batareya 7 kun yetishi shart (400 mAh, 3,7 V). Mikro-NPU, FPGA (past quvvatli) va MCU-CPU uchun energiya byudjeti va fixed-point aniqlikni hisoblaysiz.

Yuklama/model	1D-CNN 8 qatlam, 2500 nuqta
Talab	1 tahlil/10 s
Cheklov	batareya 7 kun, < 10 mVt o'rtacha
Solishtiriladi	CPU (MCU) vs NPU vs FPGA
MXU/sistolik holat	N=16
Fixed-point format	Q1.6 / Q1.14
Narx farazi	akkumulyator 1,48 Vt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU (MCU) vs NPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (1D-CNN 8 qatlam, 2500 nuqta) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (N=16) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (Q1.6 / Q1.14) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (1 tahlil/10 s) va cheklov (batareya 7 kun, < 10 mVt o'rtacha) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (akkumulyator 1,48 Vt*soat); sezgirlik tahlili.
7. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan); Manbalar ro'yxati (datasheet, MLPerf) va farazlar jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

17-variant. Neft-gaz: quvurda oqishni akustik signal bo'yicha aniqlash qiyin

Soha: Neft-gaz

Magistral quvur bo'ylab har 5 km da akustik sensor, 48 kGs signal spektrogrammasi CNN bilan tahlil qilinadi. Stansiyalar uzoqda, quvvat quyosh+akkumulyator (30 Vt), aloqa sun'iy yo'ldosh orqali qimmat. Chekkada FPGA va Jetson ni solishtirasiz.

Yuklama/model	spektrogramma CNN 128x128, 4 qatlam
Talab	har stansiya 2 tahlil/s
Cheklov	<= 30 Vt, sun'iy yo'ldosh aloqasi
Solishtiriladi	Jetson vs FPGA vs NPU
MXU/sistolik holat	N=64
Fixed-point format	Q3.12
Narx farazi	aloqa 5 $/MB

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (Jetson vs FPGA vs NPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (spektrogramma CNN 128x128, 4 qatlam) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (N=64) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (Q3.12) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (har stansiya 2 tahlil/s) va cheklov (<= 30 Vt, sun'iy yo'ldosh aloqasi) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (aloqa 5 $/MB); sezgirlik tahlili.
7. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan); Manbalar ro'yxati (datasheet, MLPerf) va farazlar jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

18-variant. Bank: chatbot uchun LLM inferens klasteri qiyin

Soha: Moliya (mijozlarga xizmat)

Bank mijozlarga 7B parametrli LLM asosidagi chatbotni ichki serverda joylashtirmoqchi: cho'qqida 200 parallel suhbat, javob tezligi kamida 15 token/s. Dekod bosqichi xotira o'tkazuvchanligi bilan cheklangani sababli HBM'li GPU, TPU va CPU ni token/s/$ bo'yicha solishtirasiz.

Yuklama/model	LLM 7B: dekod GEMV 4096x4096, prefill GEMM
Talab	200 suhbat x 15 token/s
Cheklov	ma'lumot ichkarida
Solishtiriladi	CPU vs GPU vs TPU
MXU/sistolik holat	dekod M=1..64
Fixed-point format	INT8 og'irlik / FP16
Narx farazi	3 yil, 0,10 $/kVt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs TPU + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (LLM 7B: dekod GEMV 4096x4096, prefill GEMM) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (dekod M=1..64) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (INT8 og'irlik / FP16) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (200 suhbat x 15 token/s) va cheklov (ma'lumot ichkarida) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (3 yil, 0,10 $/kVt*soat); sezgirlik tahlili.
7. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan); Manbalar ro'yxati (datasheet, MLPerf) va farazlar jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

19-variant. Kosmik monitoring: sun'iy yo'ldosh tasvirlarida ekin turlarini tasniflash qiyin

Soha: Masofaviy zondlash

Davlat agentligi har 5 kunda respublika hududining 10 m aniqlikdagi multispektral tasvirlarini (13 kanal) yangilaydi: ~45 ming kvadrat 256x256. Tasniflash 12 soatda tugashi kerak. Egalikdagi GPU, bulut TPU va FPGA ni katta batch rejimida solishtirasiz.

Yuklama/model	U-Net yengil, 13 kanal kirish
Talab	45 ming plitka / 12 soat
Cheklov	paketli, kechikish muhim emas
Solishtiriladi	GPU vs TPU vs FPGA
MXU/sistolik holat	toq K=13*9=117
Fixed-point format	BF16 / INT8
Narx farazi	5 kunda 1 ishga tushirish

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (GPU vs TPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (U-Net yengil, 13 kanal kirish) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (toq K=13*9=117) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (BF16 / INT8) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (45 ming plitka / 12 soat) va cheklov (paketli, kechikish muhim emas) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (5 kunda 1 ishga tushirish); sezgirlik tahlili.
7. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan); Manbalar ro'yxati (datasheet, MLPerf) va farazlar jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20

20-variant. Kiberxavfsizlik: zararli fayllarni bayt-ketma-ketlik bo'yicha aniqlash

qiyin

Soha: Kiberxavfsizlik

Korporativ pochta shlyuzida kuniga 1,5 mln ilova MalConv uslubidagi 1D-CNN bilan tekshiriladi, cho'qqi 60 fayl/s, kechikish 300 ms. Shlyuz 1U serverga sig'ishi kerak (bitta PCIe slot). GPU, FPGA va CPU ni slot, quvvat va TCO bo'yicha solishtirasiz.

Yuklama/model	1D-CNN embedding 8 + conv 512 filtr, 2 MB kirish
Talab	60 fayl/s, p95 <= 300 ms
Cheklov	1U, bitta PCIe, 250 Vt
Solishtiriladi	CPU vs GPU vs FPGA
MXU/sistolik holat	M=1..16
Fixed-point format	Q3.4 (8 bit)
Narx farazi	elektr 0,10 $/kVt*soat

QADAMLAR

1. 1-qadam kabi pasport jadvalini tuzing (CPU vs GPU vs FPGA + kamida yana bitta arxitektura), ridge nuqtasi va TOP/s/Vt ni hisoblang; manbalarni ko'rsating.
2. Variant yuklamasi (1D-CNN embedding 8 + conv 512 filtr, 2 MB kirish) uchun amallar va baytlar formulasini yozing, AI ni hisoblang va Roofline grafigida chegara turini (H/X) ko'rsating.
3. 2-qadam sistolik simulyatorini variant holati (M=1..16) uchun ishlating: to'g'rilik, taktlar va MXU foydalanish foizi.
4. 3- va 4-qadamlar: fixed-point (Q3.4 (8 bit)) yoki INT8 kvantlashda kichik muqobil modelning (digits/sintetik) aniqlik pasayishi va SQNR'i; NPU bo'linishi kerak bo'lsa modellang.
5. 5-qadam kabi sandbox CPU'da variant yadrosiga o'xshash GEMM/conv ni o'lchab, samaradorlikni topib, tanlangan arxitekturalarga ekstrapolyatsiya qiling.
6. 6-qadam: talab (60 fayl/s, p95 <= 300 ms) va cheklov (1U, bitta PCIe, 250 Vt) bo'yicha qurilmalar soni, quvvat, mJ/inferens va TCO (elektr 0,10 $/kVt*soat); sezgirlik tahlili.
7. 7-qadam: variant ssenariysi uchun vaznlar va qat'iy cheklovlar bilan WSM/TOPSIS tanlov matritsasi, Monte-Karlo barqarorligi va yakuniy tavsiya.

Topshiriladi: Kod (.py, har qadam alohida); Pasport va natijalar jadvallari; Roofline va TCO grafiklari (PNG); Xulosa (kamida 150 so'z, taxminiy raqamlar belgilangan); Manbalar ro'yxati (datasheet, MLPerf) va farazlar jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, Hisob-kitob va jadvallar — 25, Tahlil, tavsiya va asoslash — 25, Hisobot sifati — 20